An Explainable Fracture-Detection Prototype and an Audit of Evaluation Leakage in Publicly Sourced Radiographs: Implemented Corrections and a Leak-Free Re-Evaluation Protocol

Yug G. Lakhani

Department of Computer Engineering, Gyanmanjari Innovative University, Bhavnagar, Gujarat, India

yglakhani@gmiu.edu.in · ORCID: 0009-0004-2930-4953

Abstract

Missed fractures on radiographs are a recognised source of diagnostic error, and deep-learning decision support has reported pooled sensitivities and specificities of roughly 91–94% and 91–93% in systematic reviews. Those reviews also identify limited external validation, inconsistent evaluation, and explanations that are rarely validated quantitatively. This paper makes no new clinical-performance claim. Instead, it reports (i) an end-to-end explainable screening prototype — a fine-tuned ResNet50 with Grad-CAM, a confidence score, a manual-review flag, and a FastAPI/React web application; (ii) an audit of the prototype's original evaluation, which found that 1,268 of 1,396 validation file-name stems (90.8%) also occurred in training, consistent with augmented copies of the same radiographs crossing a random image-level split; and (iii) implemented, tested corrections: a source-grouped split using MD5, perceptual hashing and file-name stems, an annotation-verification tool that checks class IDs and bounding boxes and generates a stratified human-review sample, a dataset-registry for provenance and per-source reporting, temperature-scaling calibration with a threshold selected for a pre-declared target sensitivity, and quantitative Grad-CAM and source-confounding evaluations. The corrected pipeline was verified on a synthetic image pool (120 files derived from 40 source images collapsed to 40 groups, with no group spanning two partitions). Because the original pooled dataset, its per-source provenance, and clinician verification are not available, the earlier development-set figures are reported only as an audit record and are explicitly not estimates of generalisation. The contribution is a deployable prototype, a documented leakage audit of publicly sourced radiograph data, and a reproducible protocol that must be run on documented, verified data before any performance claim is made.

Index Terms— Fracture detection, ResNet50, transfer learning, Grad-CAM, explainable AI, data leakage, calibration, evaluation methodology, radiographs.

I. Introduction

Routine radiography is the first test for a suspected fracture, and overlooked fractures remain a known diagnostic pitfall, particularly under time pressure or when images are first read by clinicians without specialist radiology training [1], [2]. Computer-aided decision support that flags a possible fracture and shows where the model is looking could, in principle, reduce such misses. Convolutional neural networks (CNNs) are the dominant approach in medical image analysis [3], and systematic reviews of fracture detection report pooled sensitivity and specificity around 91–94% and 91–93% [1], [6], [7]. The same reviews repeatedly identify three weaknesses: scarce external validation, heterogeneous and sometimes leaky evaluation, and visual explanations that are seldom tested against expert annotation [1], [6], [7], [18].

This paper began as a conventional prototype study: fine-tune ResNet50 to classify radiographs as fracture or no fracture, explain predictions with Grad-CAM [8], and serve the result in a web application. During revision, an audit of the evaluation protocol showed that its headline result could not be trusted: the split had been made per image over a pool containing augmented copies of the same radiographs, and most validation images had a near-copy in training. Rather than replace an unverifiable number with a new one, this paper was re-scoped around what can be verified.

The contributions are:

An end-to-end explainable prototype — training, evaluation, Grad-CAM, a REST back end, a web interface and container configuration (Section IV).

A documented audit of evaluation leakage in a publicly sourced radiograph pool, including a file-name-stem overlap measurement (90.8%) and a point-by-point comparison of the original protocol against recommended safeguards (Section V).

Implemented corrections, released as code — grouped deduplicated splitting, annotation verification, provenance registration, calibration and target-sensitivity thresholding, quantitative Grad-CAM validation, and source-confounding tests (Section VI). Each tool was verified on synthetic data; none of its synthetic outputs is reported as a clinical result.

A bounded conclusion: the original accuracy is retained only as an audit record. No conclusion about generalisation, clinical performance, calibration, or explanation fidelity is drawn, and the conditions under which such conclusions could be drawn are stated exactly (Section VIII).

This positioning is deliberate. A prototype-and-audit paper is a legitimate and useful contribution: leakage of the kind found here is a documented cause of over-optimistic medical-imaging results [26], internal-only validation is known to overstate performance [19], and a worked example — including the corrected tooling — helps other groups using pooled public datasets avoid the same error.

II. Related Work

A. Prior reviews

Kalmet et al. [5] reviewed deep learning for fracture detection narratively. Kuo et al. [1], across 42 studies, reported pooled sensitivity/specificity of 92%/91% on internal and 91%/91% on external test sets, with heterogeneity attributed partly to risk of bias and fracture type. Jung et al. [6] reported pooled sensitivity 94% and specificity 92% for radiographs; Hansen et al. [7] reported 92% and 93% for wrist fractures with wide confidence intervals. Patient selection, reference standards, and scarce external validation recur as limitations.

B. Datasets

MURA [9] provides 40,561 upper-extremity radiographs labelled normal/abnormal (abnormality, not fracture specifically). FracAtlas [10] provides 4,083 radiographs with classification, localisation and segmentation annotations, 717 containing fractures; such documented, licensed datasets are the appropriate basis for external evaluation. Many influential studies instead use private institutional archives [4], [11], [12], limiting reproducibility. The present work used pooled, contributor-uploaded Kaggle datasets — a common student/research starting point, and precisely the setting in which provenance, licensing, deduplication and label conventions are easiest to lose.

C. Models

Transfer learning from ImageNet is the default response to limited labelled data. Olczak et al. [4] reported 83% accuracy, comparable to two senior surgeons; Kim and MacKinnon [11] reported AUC 0.954 retraining Inception v3 on lateral wrist radiographs; Wang et al. [12] combined ResNet50 with squeeze-and-excitation blocks for ankle fractures (93% accuracy, AUC 0.95) and used Grad-CAM qualitatively. Reported accuracies elsewhere span 92.96% for a four-network humerus ensemble [13] to 99% for MobileNet features with gradient boosting [15]; differing datasets and protocols make cross-study comparison unreliable, which motivates reporting this paper's own numbers only within their audited limits.

D. Explainability

Grad-CAM [8] weights final convolutional feature maps by class-score gradients to produce a coarse localisation map without architectural changes. In fracture work it has been used mostly qualitatively [12]. On chest radiographs, Saporta et al. [18] benchmarked saliency methods against expert segmentations and found localisation considerably worse than human annotators, especially for small or complex findings. Explanations should therefore be measured, not merely displayed — hence the quantitative protocol in Section VI.

E. Validation and clinical use

Internal findings can be optimistic: a weighted mean AUC for ankle-fracture classification fell from 0.95 internally to 0.86 externally [19]. Conversely, a multi-reader study found that AI support improved reader sensitivity by about 10 percentage points without prolonging reading time [2], supporting decision support — with the clinician retaining responsibility — as the realistic deployment model, and the only one contemplated here.

III. Research Gap and Questions

The literature shows five gaps: (G1) cross-anatomy generalisability and dataset diversity; (G2) external validation and bias control; (G3) inconsistent evaluation and reporting, including augmented evaluation data and accuracy-only reporting; (G4) unvalidated explanations; (G5) few deployable systems combining detection with explanation. This work addresses G5 (a working prototype), and contributes to G3 by auditing its own protocol, quantifying the leakage, and releasing corrected tooling. It does not claim to close G1, G2 or G4; Section VIII states what closing them requires.

RQ1. What did the original evaluation actually measure, and how far can it be trusted?

RQ2. Can a classifier, its Grad-CAM explanations and an uncertainty flag be served together as a web application?

RQ3. Which recommended safeguards did the original protocol miss, what corrections have been implemented, and what evidence is still required before a performance claim?

IV. The Prototype

A. Data used for the original prototype

Images were labelled by a preparation script: an image with a matching non-blank annotation text file was labelled fracture; otherwise no fracture. The class IDs inside the annotation files were not checked against each source dataset's definitions, and no manual spot-check was performed, so labels are unverified and their error rate is unknown. Images came from publicly available multi-source Kaggle datasets uploaded by individual contributors. Many file names follow the Roboflow export pattern (source name plus a .rf.<hash> suffix), in which one source image is exported as several augmented copies. Dataset names, URLs, licences and anatomical regions were not recorded at download, and images were not redistributed. These facts are stated here rather than in a limitations appendix because they determine what the original results mean (Section V). The final folders contained 12,255 training and 1,620 validation images (Table I).

TABLE I. Original dataset composition

Split	Fracture	No fracture	Total
Training	6,102	6,153	12,255
Validation	728	892	1,620

B. Preprocessing and augmentation

Images are converted to 3-channel RGB, resized to 224×224, and normalised with ImageNet means (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225). Training used random rescaling (uniform in [0.85, 1.0]), cropping, and horizontal flipping; validation and inference used resizing only. Truncated files were ignored by the loader.

C. Model

ResNet50 [20] pretrained on ImageNet [21], with the final layer replaced by a two-unit linear layer (fracture = 0, no fracture = 1); all layers were fine-tuned. Softmax gives class probabilities; the arg-max is the prediction and its probability is displayed as confidence. Predictions below 0.80 confidence are flagged for manual review. That threshold was a safety convention — untuned and unvalidated — and the softmax outputs were not calibrated; Section VI replaces this mechanism.

D. Training

PyTorch [23], Adam [22] (lr 1e-4, weight decay 1e-5), cross-entropy loss, batch size 4 (a CPU-only machine with 7.6 GB RAM), at most 10 epochs, ReduceLROnPlateau on validation loss (factor 0.3, patience 1), early stopping with patience 3, and checkpoint selection by lowest validation loss. No seed was fixed and each configuration ran once. All parameters, including BatchNorm [24], were trainable at batch size 4, which gives noisy normalisation statistics [25]; this was not tested and is not invoked to explain any result.

TABLE II. Original training configuration

Setting	Value
Backbone	ResNet50, ImageNet-pretrained
Input	224 × 224 × 3
Optimiser / loss	Adam, lr 1e-4, wd 1e-5 / cross-entropy
Batch size / max epochs	4 / 10
Scheduler / early stopping	ReduceLROnPlateau (0.3, patience 1) / patience 3 on validation loss

Setting	Value
Checkpoint rule	Lowest validation loss
Hardware	CPU only, 7.6 GB RAM; run time not recorded

E. Grad-CAM

Grad-CAM [8] was applied at the last residual stage (layer4), the same target as the deployed service: class-score gradients are spatially averaged to weight each channel, the weighted activation sum is passed through ReLU, upscaled to 224×224 and max-normalised to [0, 1].

F. Web application

FastAPI/Uvicorn serves GET /, GET /health, and POST /predict, which accepts an image and an include_heatmap flag and returns the prediction, confidence, and — when Grad-CAM is initialised — the heat map. The React (Vite, Tailwind) front end uploads an image and displays the result under "Grad-CAM Explanation". Backend and frontend are containerised with Docker Compose; the >90 MB checkpoint is stored with Git LFS. This answers RQ2 affirmatively at prototype level. The system is a research prototype, not a diagnostic device.

G. Original evaluation protocol

The validation folder was evaluated with accuracy, sensitivity, specificity, precision, negative predictive value, F1, balanced accuracy, MCC and AUC (fracture positive). Critically, the same folder drove LR scheduling, early stopping and checkpoint selection: it was a development set, not a test set. No confidence intervals were reported.

V. Audit of the Original Evaluation (RQ1, RQ3)

A. What was measured

Training ran eight epochs; the selected checkpoint (epoch 5, validation loss 0.2119) recorded validation accuracy 94.07%, with a confusion matrix of 675 true positives, 53 false negatives, 43 false positives and 849 true negatives (sensitivity 92.72%, specificity 95.18%, precision 94.01%, F1 93.36%, AUC 0.9813). Ten inference samples all returned confidence between 0.9975 and 1.0000 — an illustration of uncalibrated softmax output, not of reliability.

These numbers are exactly reproducible from the stored run metadata, and they are reported here for transparency. They are not performance estimates, for the two reasons below.

B. Finding 1 — no independent test set

The reported accuracy is identical to the epoch-5 validation accuracy used for model selection. No image set was held back from scheduling, stopping and selection, so even a perfectly clean split would yield an optimistic estimate.

C. Finding 2 — augmented-copy leakage across the split

The split was random per image over a pool containing augmented copies (names showing e.g. repeated rotation suffixes and .rf. exports). Comparing file-name stems (the name before .rf.<hash>) between folders: the validation set contains 1,396 stems, of which 1,268 (90.8%) also occur in training; only 128 validation stems have no training counterpart. The model was therefore evaluated largely on images it had effectively seen. This check is name-based — it neither proves pixel identity nor finds renamed copies — but at 90.8% it is sufficient to invalidate the validation set as a measure of performance on new radiographs. The size of the resulting optimism was not, and from the retained data cannot be, measured. Duplicate images across pooled public datasets are a documented leakage source [26].

D. Protocol audit

TABLE III. Recommended practice vs. original protocol, and the correction implemented in this revision

Recommended practice	Original protocol	Correction implemented (Section VI)
Patient/source-level split	Per-image random split	Grouped split by MD5 + perceptual hash + stem

Recommended practice	Original protocol	Correction implemented (Section VI)
Independent held-out test	None (validation used for selection)	Train/validation/test 70/15/15 over groups
External validation	None	External-set evaluation entry point; dataset to be registered and licensed
Verified labels and boxes	Blank-file rule; class IDs unchecked	Annotation verifier + stratified human-review sample
Provenance and licences	Not recorded	Dataset registry + per-image inventory, per-source reporting
Calibration; threshold at target sensitivity	Fixed 0.5; untuned 0.80 flag	Temperature scaling; threshold chosen on validation for pre-declared sensitivity
Quantitative explanation validation	One qualitative example	Pointing game, energy-in-box, IoU, randomisation sanity check
Source-confounding control	Not examined	Per-source spread, leave-one-source-out, source predictability, border/marker ablation
Repeated runs / fixed seeds	Single run, no seed	Seeded multi-run training and aggregation tooling

E. Answer to RQ1

The original evaluation measured how well a checkpoint selected on a development set performed on that same, leakage-contaminated set. Its figures are development diagnostics only. They must not be compared with systems evaluated on independent data, and no such comparison is made anywhere in this paper.

VI. Implemented Corrections

All corrections below are implemented in the accompanying code repository and were verified on synthetic data. They are described as protocol and tooling, because the original pooled images and their provenance are unavailable for a re-run; claiming re-trained results without that data would repeat the original error in a new form.

A. Leak-free grouped split

scripts/make_split.py builds groups with union–find over three identity relations: identical MD5; 64-bit perceptual hash (32×32 greyscale DCT, top-left 8×8) within a Hamming distance of 4 bits; and equal file-name stem before the .rf. suffix. Groups are partitioned 70/15/15 with seeded GroupShuffleSplit, and the script asserts that no group spans two partitions. It writes a manifest (image, label, group, split, source dataset) and a per-partition, per-source summary.

Verification. On a synthetic pool of 120 files generated from 40 source images (three exported copies each), grouping recovered exactly 40 groups and the overlap assertion passed. This verifies the plumbing, not any clinical quantity.

B. Annotation verification

scripts/verify_annotations.py parses every YOLO annotation file, requires class IDs to exist in a per-source mapping verified against that source's documentation, checks that boxes are normalised and in-bounds, and reports every violation by file and line. It then draws a deterministic, class-stratified random sample and writes a review sheet (auto-label, boxes, and blank columns for reviewer label, box judgement, identity, role and date). The rule "non-blank file means fracture" is thereby replaced by an auditable check. Human — ideally clinician — completion of that sheet remains a human task; this paper claims the tooling and the protocol, not a verification outcome.

C. Provenance and per-source reporting

datasets/dataset_registry.csv records, per source: name, URL, licence, download date, anatomical regions, image and class counts, annotation format, and patient-ID availability. A per-image inventory links every pooled image to one registry row, and the source identifier is carried through the split manifest and prediction files into per-source metrics in every evaluation report. For the original pool this record was not retained and cannot be reconstructed; the registry therefore also serves as an explicit statement of what is unknown about the original data, and the images are not redistributed.

D. Independent and external evaluation

scripts/train.py fixes seeds, selects the checkpoint on validation F1 only, evaluates the held-out test partition once, saves the checkpoint and per-image prediction files, and accepts an external ImageFolder that is evaluated with the frozen checkpoint and never used for selection. scripts/aggregate.py summarises repeated seeded runs as mean ± SD. A documented external dataset with a licence permitting the use (for example FracAtlas [10]) is the intended external set; selecting, downloading and registering it is part of the re-evaluation protocol, and no external figure is reported until that is done.

E. Calibration and target-sensitivity thresholding

scripts/calibrate_threshold.py fits a single temperature on validation logits by minimising negative log-likelihood, chooses — on calibrated validation probabilities — the threshold achieving a pre-declared target sensitivity of 0.95 with the highest specificity, and freezes both quantities before test or external use. It reports ECE (15 bins), Brier score, the full metric set per source, and group-cluster bootstrap confidence intervals. This replaces the untuned 0.5/0.80 conventions. (On synthetic predictions used to test the tool, ECE fell from 0.152 before to 0.019 after scaling; the figure verifies the implementation only.)

F. Quantitative Grad-CAM validation

scripts/gradcam_eval.py scores every boxed fracture image with: the pointing game (CAM maximum inside the box); CAM energy inside the box, reported alongside the box-area fraction that defines chance; IoU of the CAM binarised at 0.5 of its maximum; and, given heat maps from a weight-randomised model, the Spearman correlation between original and randomised maps (a faithful explanation must change under randomisation [31]). Until this is run against verified boxes, Grad-CAM in the application is presented as an aid to inspecting model behaviour — not as segmentation, localisation evidence, or proof of a correct prediction, consistent with the benchmarking evidence in [18].

G. Source-confounding tests

scripts/source_confounding.py reports per-source sensitivity spread, leave-one-source-out estimates, the accuracy with which the source dataset can be predicted from image embeddings (cross-validated, against the majority-class chance level — a shortcut-risk signal in the sense of [27], [28]), and a paired border/text-marker ablation with McNemar's test. These tests target the realistic failure mode of pooled public data: a model keying on borders, markers or acquisition style rather than bone.

VII. Discussion

Relation to the literature. The original development-set sensitivity (92.7%) and specificity (95.2%) sit numerically inside the pooled ranges of [1], [6], [7]. No inference is drawn from that coincidence: the literature estimates derive in part from external test sets, and this paper's derive from a contaminated development set. The finding that survives audit is methodological — augmented-copy leakage was present in publicly sourced radiograph data, at 90.8% stem overlap, and was detectable with a trivial file-name check that costs nothing to run before training.

Error profile, honestly stated. On the contaminated set the model missed 7.3% of fractures and flagged 4.8% of normal images; both rates are optimistic by an unknown amount. For a screening aid, missed fractures are the costlier error, which is why the corrected protocol selects its operating point at a target sensitivity on clean validation data rather than defaulting to 0.5. The rising validation loss after epoch 5 alongside 99.6% final training accuracy indicates overfitting; whether regularisation, frozen BatchNorm statistics or a larger batch would help is untested and is not asserted.

Role of explanation. A heat map that has never been scored against a box is a user-interface feature, not evidence. The corrected protocol therefore treats Grad-CAM as a hypothesis about model behaviour, to be tested with the pointing game, energy and IoU metrics and the randomisation check before any localisation language is used.

Clinical positioning. Reader-study evidence that AI support can raise sensitivity [2] belongs to the cited literature, not to this system: no reader study, latency measurement, or prospective evaluation has been performed. The application is a

research prototype supporting a decision-support workflow in principle only.

VIII. Limitations and Conditions for a Performance Claim

The original results come from one pooled, undocumented dataset; there is no independent test set or external evaluation behind them, and the split was per image, with 90.8% train–validation stem overlap. The estimates are optimistically biased by an unknown amount.

Labels were inferred from annotation-file presence; class IDs were not verified at the time and no clinician review exists, so both false "fracture" and false "normal" labels are possible at unknown rates.

Source names, URLs, licences and anatomical regions were not retained; the dataset cannot be reconstructed from this paper, and per-source and per-region performance is unknown.

The task is binary; the model neither localises nor grades fractures.

Training was a single unseeded run at batch size 4 with trainable BatchNorm; run-to-run variability and that design choice's effect are unknown.

Probabilities were uncalibrated; the 0.5 operating point and 0.80 review flag were conventions, and inference latency was not measured.

Grad-CAM was shown for one correctly classified example only, on an image containing rotation padding and a text marker; acquisition artefacts are present in the pool and shortcut reliance was not excluded.

The corrections in Section VI are implemented and synthetically verified, but they have not yet produced new empirical results, because the underlying images, provenance and human verification are unavailable. This paper therefore claims a prototype, an audit, and a protocol — nothing more.

A performance claim becomes permissible when, and only when: the data are documented in the registry with licences; annotations pass the class-ID audit and a completed, attributed human spot-check; the grouped split's integrity assertion passes on the real pool; the threshold and temperature are frozen from validation; test and registered external results are reported per source with group-resampled intervals; and Grad-CAM and source-confounding metrics are reported alongside accuracy. Section VI's commands, run in order on such data, produce exactly those artefacts.

IX. Conclusion

An explainable fracture-screening prototype — ResNet50, Grad-CAM, confidence and review flag in a containerised web application — was built and its original evaluation was audited. The audit found no independent test set and 90.8% augmented-copy overlap between training and validation, so the prototype's 94.07% development-set accuracy cannot be read as real-world or clinical performance. Corrections for every identified defect have been implemented and verified as software: grouped perceptual-hash deduplication and splitting, annotation verification with a human-review workflow, provenance registration with per-source reporting, calibration with a target-sensitivity threshold, and quantitative explanation and confounding evaluation. The immediate next step is a single re-run of that protocol on documented, verified data with a registered external set. Until then, the contribution of this work is the prototype, the leakage audit, and a reproducible route to a claim that can be believed.

Data and Code Availability

Source code (original training and deployment, plus all Section VI tools), the dataset-registry and review-sheet templates, and run metadata are available at https://github.com/Unusualyug/fracture_detection_. The model checkpoint is stored with Git LFS in the same repository. Images were obtained from publicly available Kaggle datasets whose names, URLs and licences were not retained; they are not redistributed, both because the licence terms are unknown and because the dataset cannot be exactly reproduced. No patient-identifiable information was noticed in the files, although this was not checked systematically. No ethical approval was sought: no patient was contacted and only publicly available images were used. The system is a research prototype and is not intended for clinical use.

References

[1] R. Y. L. Kuo et al., "Artificial intelligence in fracture detection: a systematic review and meta-analysis," Radiology, vol. 304, no. 1, pp. 50–62, 2022, doi: 10.1148/radiol.211785.

[2] A. Guermazi et al., "Improving radiographic fracture recognition performance and efficiency using artificial intelligence," Radiology, vol. 302, no. 3, pp. 627–636, 2022, doi: 10.1148/radiol.210937.

[3] G. Litjens et al., "A survey on deep learning in medical image analysis," Medical Image Analysis, vol. 42, pp. 60–88, 2017, doi: 10.1016/j.media.2017.07.005.

[4] J. Olczak et al., "Artificial intelligence for analyzing orthopedic trauma radiographs: deep learning algorithms—are they on par with humans for diagnosing fractures?," Acta Orthopaedica, vol. 88, no. 6, pp. 581–586, 2017, doi: 10.1080/17453674.2017.1344459.

[5] P. H. S. Kalmet et al., "Deep learning in fracture detection: a narrative review," Acta Orthopaedica, vol. 91, no. 2, pp. 215–220, 2020, doi: 10.1080/17453674.2019.1711323.

[6] J. Jung, J. Dai, B. Liu, and Q. Wu, "Artificial intelligence in fracture detection with different image modalities and data types: a systematic review and meta-analysis," PLOS Digital Health, vol. 3, no. 1, e0000438, 2024, doi: 10.1371/journal.pdig.0000438.

[7] V. Hansen, J. Jensen, M. W. Kusk, O. Gerke, H. B. Tromborg, and S. Lysdahlgaard, "Deep learning performance compared to healthcare experts in detecting wrist fractures from radiographs: a systematic review and meta-analysis," European Journal of Radiology, vol. 174, Art. no. 111399, 2024, doi: 10.1016/j.ejrad.2024.111399.

[8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 618–626.

[9] P. Rajpurkar et al., "MURA: Large dataset for abnormality detection in musculoskeletal radiographs," arXiv:1712.06957, 2017.

[10] I. Abedeen et al., "FracAtlas: A dataset for fracture classification, localization and segmentation of musculoskeletal radiographs," Scientific Data, vol. 10, Art. no. 521, 2023, doi: 10.1038/s41597-023-02432-4.

[11] D. H. Kim and T. MacKinnon, "Artificial intelligence in fracture detection: transfer learning from deep convolutional neural networks," Clinical Radiology, vol. 73, no. 5, pp. 439–445, 2018, doi: 10.1016/j.crad.2017.11.015.

[12] H. Wang, J. Ying, J. Liu, T. Yu, and D. Huang, "Harnessing ResNet50 and SENet for enhanced ankle fracture identification," BMC Musculoskeletal Disorders, vol. 25, Art. no. 250, 2024, doi: 10.1186/s12891-024-07355-8.

[13] A. Tahir, A. Saadia, K. Khan, A. Gul, A. Qahmash, and R. N. Akram, "Enhancing diagnosis: ensemble deep-learning model for fracture detection using X-ray images," Clinical Radiology, vol. 79, no. 11, pp. e1394–e1402, 2024, doi: 10.1016/j.crad.2024.08.006.

[14] T. Aldhyani et al., "Diagnosis and detection of bone fracture in radiographic images using deep learning approaches," Frontiers in Medicine, vol. 11, Art. no. 1506686, 2025, doi: 10.3389/fmed.2024.1506686.

[15] A. Alam et al., "Novel transfer learning based bone fracture detection using radiographic images," BMC Medical Imaging, vol. 25, Art. no. 5, 2025, doi: 10.1186/s12880-024-01546-4.

[16] M. He, X. Wang, and Y. Zhao, "A calibrated deep learning ensemble for abnormality detection in musculoskeletal radiographs," Scientific Reports, vol. 11, Art. no. 9097, 2021, doi: 10.1038/s41598-021-88578-w.

[17] Y. L. Thian et al., "Convolutional neural networks for automated fracture detection and localization on wrist radiographs," Radiology: Artificial Intelligence, vol. 1, no. 1, Art. no. e180001, 2019, doi: 10.1148/ryai.2019180001.

[18] A. Saporta et al., "Benchmarking saliency methods for chest X-ray interpretation," Nature Machine Intelligence, vol. 4, pp. 867–878, 2022.

[19] J. Olczak et al., "External validation of an artificial intelligence multi-label deep learning model capable of ankle fracture classification," BMC Musculoskeletal Disorders, vol. 25, Art. no. 788, 2024, doi: 10.1186/s12891-024-07884-2.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[21] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[23] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[24] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. Int. Conf. Machine Learning (ICML), 2015, pp. 448–456.

[25] Y. Wu and K. He, "Group normalization," in Proc. European Conf. Computer Vision (ECCV), 2018, pp. 3–19.

[26] M. Roberts et al., "Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans," Nature Machine Intelligence, vol. 3, pp. 199–217, 2021.

[27] J. R. Zech et al., "Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: a cross-sectional study," PLOS Medicine, vol. 15, no. 11, Art. no. e1002683, 2018.

[28] A. J. DeGrave, J. D. Janizek, and S.-I. Lee, "AI for radiographic COVID-19 detection selects shortcuts over signal," Nature Machine Intelligence, vol. 3, pp. 610–619, 2021.

[29] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. Int. Conf. Machine Learning (ICML), 2017, pp. 1321–1330.

[30] J. Zhang, S. A. Bargal, Z. Lin, J. Brandt, X. Shen, and S. Sclaroff, "Top-down neural attention by excitation backprop," Int. J. Computer Vision, vol. 126, pp. 1084–1102, 2018.

[31] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018.